

Memory Management Address Translation

CS232 Operating Systems

Fall 2019

Taj Muhammad Khan

Control and Efficiency

- Control
 - Restrict the process to only its own memory
- Efficiency
 - Share the memory between different processes
 - Do it with minimum cost
- Address Translation
 - Needs hardware support
 - Needs OS support

Assumption

- A process address space lies contiguously in memory
- Size of a process address space is smaller than physical memory
- All process address spaces are of same size

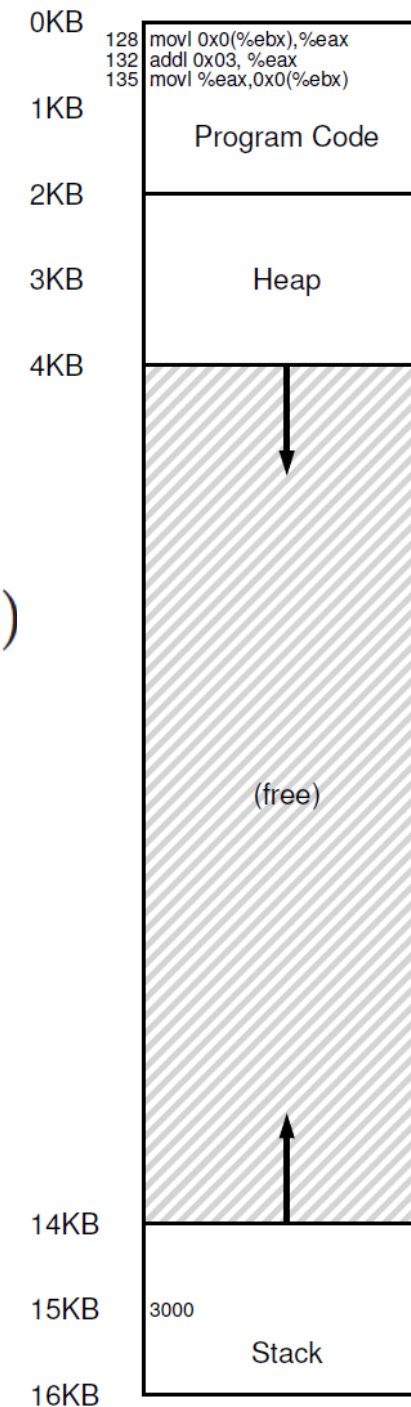
Example

```
void func() {  
    int x = 3000; // thanks, Perry.  
    x = x + 3;    // this is the line of code we are interested in  
    ...  
}
```

128: movl 0x0(%ebx), %eax	;load 0+ebx into eax
132: addl \$0x03, %eax	;add 3 to eax register
135: movl %eax, 0x0(%ebx)	;store eax back to mem

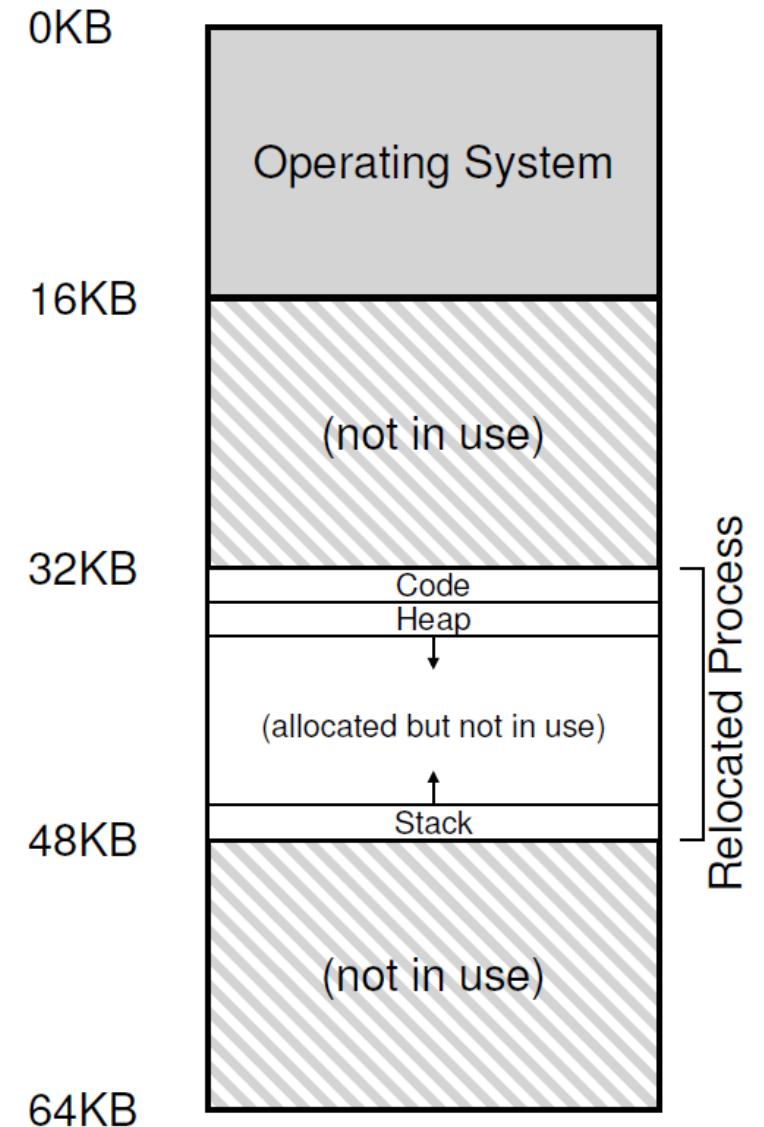
Memory accesses

- Fetch instruction at address 128
- Execute this instruction (load from address 15 KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)



Virtualization

- From the process perspective, its address space starts at 0
- In reality the OS might not place the process image at that address
- How can the OS provide this illusion in a manner transparent to the process



Dynamic relocation

- Two special registers in CPU
 - Base
 - Bound (limit)
- Every address generated by the program is translated by the hardware:

`physical address = virtual address + base`

- Example `128: movl 0x0(%ebx), %eax`

Memory Management Unit (MMU)

- Base register
 - Contains the start of process address space in physical memory
 - Used in address translation
- Bound register
 - Can contain either:
 - The size of process address space, or
 - The final physical address of the process address space
 - Used in checking for illegal memory accesses

H/W support we need for MM

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support address translation and bounds checks</i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i>OS must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i>OS must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or out-of-bounds memory</i>

OS support we need for MM

OS Requirements	Notes
Memory management	<i>Need to allocate memory for new processes; Reclaim memory from terminated processes; Generally manage memory via free list</i>
Base/bounds management	<i>Must set base/bounds properly upon context switch</i>
Exception handling	<i>Code to run when exceptions arise; likely action is to terminate offending process</i>

Can the OS move a process image in physical memory?

Putting it all together

- Show Figure 15.5 OSTEP

Issues

- Process layout is contiguous in physical memory.
 - Internal fragmentation (space wasted *inside* allotted unit)